

1) Prompt Chaining?

Code:

```
import os
import time
from google import genai

# Create Gemini client
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

MODEL = "gemini-3.6-flash"

# =====
# FUNCTION TO CALL GEMINI WITH RETRY
# =====

def generate_response(prompt):
    for attempt in range(3):
        try:
            response = client.models.generate_content(
                model=MODEL,
                contents=prompt
            )
            return response.text

        except Exception as e:
            print(f"\nAPI error. Attempt {attempt + 1}/3")
            print(e)

            if attempt < 2:
                print("Retrying in 5 seconds...")
                time.sleep(5)
            else:
                raise

# =====
# INPUT TEXT
# =====

text = """
Artificial Intelligence is a branch of computer science that focuses on
creating systems capable of performing tasks that normally require human
intelligence. AI is used in healthcare, education, finance, transportation,
agriculture and many other industries. Machine learning is a major part of
AI that allows systems to learn patterns from data. Deep learning uses neural
networks with multiple layers to solve complex problems. Generative AI can
create text, images, audio and code. As AI continues to develop, organizations
need to consider issues such as privacy, security, bias and responsible use.
```

"""

```
# =====  
# STEP 1: EXTRACT IMPORTANT INFORMATION  
# =====
```

```
prompt1 = f"""  
Read the following text and extract the most important information.
```

```
Text:  
{text}
```

```
Give the important facts and concepts as bullet points.
```

"""

```
key_points = generate_response(prompt1)
```

```
print("\n=====  
print("STEP 1: KEY POINTS")  
print("=====  
print(key_points)
```

```
# =====  
# STEP 2: ORGANIZE INFORMATION  
# =====
```

```
prompt2 = f"""  
Organize the following information into logical categories.
```

```
Information:  
{key_points}
```

```
Create 3 to 5 categories and place the relevant points under each category.
```

"""

```
organized_points = generate_response(prompt2)
```

```
print("\n=====  
print("STEP 2: ORGANIZED INFORMATION")  
print("=====  
print(organized_points)
```

```
# =====  
# STEP 3: GENERATE SUMMARY  
# =====
```

```
prompt3 = f"""  
Using the organized information below, write a concise summary.
```

```
Information:  
{organized_points}
```

Requirements:

- Use simple English
- Include the major concepts
- Do not add information that is not present
- Keep the summary between 100 and 150 words

"""

```
summary = generate_response(prompt3)
```

```
print("\n=====")
print("STEP 3: SUMMARY")
print("=====")
print(summary)
```

```
# =====
# STEP 4: IMPROVE THE SUMMARY
# =====
```

```
prompt4 = f"""
Improve the following summary.
```

```
Summary:
{summary}
```

Requirements:

- Correct grammar
- Remove unnecessary repetition
- Keep important information
- Use clear and simple language
- Keep it below 100 words

"""

```
final_summary = generate_response(prompt4)
```

```
print("\n=====")
print("STEP 4: FINAL SUMMARY")
print("=====")
print(final_summary)
```

```
# =====
# EXPERIMENT COMPLETED
# =====
```

```
print("\n=====")
print("PROMPT CHAINING EXPERIMENT COMPLETED")
print("=====")
```

OUTPUT:

```
(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026> $env:GEMINI_API_KEY="AQ.Ab8RN6LcP57Qq4a8kSJFE9q2YQ3Ad8dMIB6Y3Jm_2lPGsmMuRg"
(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026> echo $env:GEMINI_API_KEY
AQ.Ab8RN6LcP57Qq4a8kSJFE9q2YQ3Ad8dMIB6Y3Jm_2lPGsmMuRg
(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026> python prompt-chaining.py
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC
in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommen
d to use AFC in Chat.send_message_stream.
```

STEP 1: KEY POINTS

Here are the key facts and concepts extracted from the text:

- * **Definition of Artificial Intelligence (AI):** A branch of computer science focused on creating systems that can perform tasks that typically require human intelligence.
- * **Industry Applications:** AI is utilized across various fields, including healthcare, education, finance, transportation, and agriculture.
- * **Machine Learning:** A major component of AI that enables systems to identify and learn patterns from data.
- * **Deep Learning:** A technique that uses multi-layered neural networks to solve complex problems.
- * **Generative AI:** A type of AI capable of creating text, images, audio, and code.
- * **Organizational Considerations:** As AI advances, organizations must address key issues such as privacy, security, bias, and responsible use.

STEP 2: ORGANIZED INFORMATION

Here is the information organized into **4 logical categories**:

1. Fundamental Concepts

- * **Definition of Artificial Intelligence (AI):** A branch of computer science focused on creating systems that can perform tasks that typically require human intelligence.

2. AI Technologies and Techniques

- * **Machine Learning:** A major component of AI that enables systems to identify and learn patterns from data.
- * **Deep Learning:** A technique that uses multi-layered neural networks to solve complex problems.
- * **Generative AI:** A type of AI capable of creating text, images, audio, and code.

3. Real-World Applications

- * **Industry Applications:** AI is utilized across various fields, including healthcare, education, finance, transportation, and agriculture.

4. Ethics and Governance

- * **Organizational Considerations:** As AI advances, organizations must address key issues such as privacy, security, bias, and responsible use.

STEP 3: SUMMARY

Artificial Intelligence (AI) is a branch of computer science focused on creating systems that perform tasks requiring human intelligence. Key AI technologies include Machine Learning, Deep Learning, and Generative AI. Machine Learning enables systems to identify and learn patterns from data. Deep Learning uses multi-layered neural networks to solve complex problems. Generative AI is capable of creating content such as text, images, audio, and code.

Today, AI is utilized across various real-world fields, including healthcare, education, finance, transportation, and agriculture. As AI technology advances, organizations must address key issues related to ethics and governance. These critical considerations include privacy, security, bias, and responsible use.

STEP 4: FINAL SUMMARY

Improved Summary:

Artificial Intelligence (AI) creates computer systems capable of performing human-like tasks. Key technologies include Machine Learning (finding data patterns), Deep Learning (solving complex problems via neural networks), and Generative AI (creating text, media, and code).

Today, AI is applied across healthcare, finance, education, transportation, and agriculture. As the technology advances, organizations must prioritize ethics and governance to address critical issues like privacy, security, bias, and responsible use.

(68 words)

PROMPT CHAINING EXPERIMENT COMPLETED

```
(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026>
* History restored
```

2)Sql Agent ?

Code:

```
import sqlite3

# =====
# CREATE DATABASE
# =====

connection = sqlite3.connect("students.db")
cursor = connection.cursor()

# =====
# CREATE TABLE
# =====

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
    marks INTEGER
)
""")

# =====
# INSERT SAMPLE DATA
# =====

cursor.execute("DELETE FROM students")

students = [
    (1, "Rahul", "CSE", 95),
    (2, "Priya", "CSE", 88),
    (3, "Arjun", "ECE", 91),
    (4, "Sneha", "IT", 84),
    (5, "Kiran", "CSE", 78)
]

cursor.executemany(
    "INSERT INTO students VALUES (?, ?, ?, ?)",
    students
)

connection.commit()

# =====
# DATABASE TOOL
# =====
```

```

def execute_sql(query):

    try:
        cursor.execute(query)
        return cursor.fetchall()

    except Exception as e:
        return f"Database Error: {e}"

# =====
# USER QUESTION
# =====

question = input(
    "\nEnter your question about the students database: "
)

# =====
# REACT - REASON
# =====

print("\n=====")
print("REACT AGENT")
print("=====")

print("\nTHOUGHT:")
print("I need to understand the user's question and find the required information.")

# =====
# REACT - ACTION
# =====

question_lower = question.lower()

if "highest" in question_lower or "maximum" in question_lower:
    sql_query = """
SELECT name, marks
FROM students
ORDER BY marks DESC
LIMIT 1;
"""

elif "lowest" in question_lower or "minimum" in question_lower:
    sql_query = """
SELECT name, marks
FROM students
ORDER BY marks ASC
LIMIT 1;
"""

elif "average" in question_lower:

```

```
    sql_query = ""
SELECT AVG(marks)
FROM students;
""
```

```
elif "all students" in question_lower or "show students" in question_lower:
```

```
    sql_query = ""
SELECT * FROM students;
""
```

```
elif "cse" in question_lower:
```

```
    sql_query = ""
SELECT * FROM students
WHERE department = 'CSE';
""
```

```
else:
```

```
    sql_query = ""
SELECT * FROM students;
""
```

```
print("\nACTION:")
print("execute_sql")
```

```
print("\nSQL QUERY:")
print(sql_query)
```

```
# =====
# TOOL EXECUTION
# =====
```

```
result = execute_sql(sql_query)
```

```
# =====
# REACT - OBSERVATION
# =====
```

```
print("\nOBSERVATION:")
print(result)
```

```
# =====
# FINAL ANSWER
# =====
```

```
if isinstance(result, str):
```

```
    final_answer = result
```

```
elif "highest" in question_lower or "maximum" in question_lower:
```

```
    if result:
```

```

    name = result[0][0]
    marks = result[0][1]

    final_answer = (
        f"{name} has the highest marks with {marks} marks."
    )
else:
    final_answer = "No student data found."

elif "lowest" in question_lower or "minimum" in question_lower:

    if result:
        name = result[0][0]
        marks = result[0][1]

        final_answer = (
            f"{name} has the lowest marks with {marks} marks."
        )
    else:
        final_answer = "No student data found."

elif "average" in question_lower:

    if result:
        average = result[0][0]

        final_answer = (
            f"The average marks of the students is {average:.2f}."
        )
    else:
        final_answer = "No student data found."

else:

    final_answer = f"The database returned: {result}"

print("\n=====")
print("FINAL ANSWER")
print("=====")

print(final_answer)

# =====
# CLOSE DATABASE
# =====

connection.close()

print("\n=====")
print("SQL AGENT EXPERIMENT COMPLETED")
print("=====")

```

OUTPUT:

```
(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026> python sql_agent.py

Enter your question about the students database: What is the average marks?

=====
REACT AGENT
=====

THOUGHT:
I need to understand the user's question and find the required information.

ACTION:
execute_sql

SQL QUERY:

SELECT AVG(marks)
FROM students;

OBSERVATION:
[(87.2,)]

=====
FINAL ANSWER
=====
The average marks of the students is 87.20.

=====
SQL AGENT EXPERIMENT COMPLETED
=====

(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026> python sql_agent.py

Enter your question about the students database: Show all students

Enter your question about the students database: Show all students

=====
REACT AGENT
=====

THOUGHT:
I need to understand the user's question and find the required information.

ACTION:
execute_sql

SQL QUERY:

SELECT * FROM students;

OBSERVATION:
[(1, 'Rahul', 'CSE', 95), (2, 'Priya', 'CSE', 88), (3, 'Arjun', 'ECE', 91), (4, 'Sneha', 'IT', 84), (5, 'Kiran', 'CSE', 78)]

=====
FINAL ANSWER
=====
The database returned: [(1, 'Rahul', 'CSE', 95), (2, 'Priya', 'CSE', 88), (3, 'Arjun', 'ECE', 91), (4, 'Sneha', 'IT', 84), (5, 'Kiran', 'CSE', 78)]

=====
SQL AGENT EXPERIMENT COMPLETED
=====

(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026> python sql_agent.py

Enter your question about the students database: Show CSE students

=====
REACT AGENT
=====

THOUGHT:
```

```

THOUGHT:
I need to understand the user's question and find the required information.

ACTION:
execute_sql

SQL QUERY:

SELECT * FROM students
WHERE department = 'CSE';

OBSERVATION:
[(1, 'Rahul', 'CSE', 95), (2, 'Priya', 'CSE', 88), (5, 'Kiran', 'CSE', 78)]

=====
FINAL ANSWER
=====
The database returned: [(1, 'Rahul', 'CSE', 95), (2, 'Priya', 'CSE', 88), (5, 'Kiran', 'CSE', 78)]

=====
SQL AGENT EXPERIMENT COMPLETED
=====
(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026>
* History restored

```

3) Visual qa ?

Code:

```

import os
from google import genai

# Create Gemini client
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

# =====
# IMAGE RETRIEVAL / VISUAL QA SYSTEM
# =====

image_path = "image.jpg"

# Read image
with open(image_path, "rb") as f:
    image_data = f.read()

# Question about the image
question = """
Describe what is visible in this image.
Identify the main objects and explain what is happening.
"""

# Send image + question to Gemini
response = client.models.generate_content(
    model="gemini-3.6-flash",
    contents=[
        {

```

```

        "inline_data": {
            "mime_type": "image/jpeg",
            "data": image_data
        }
    },
    question
]
)

# Display answer
print("\n=====")
print("VISUAL QA SYSTEM")
print("=====")

print("\nQuestion:")
print(question)

print("\nAnswer:")
print(response.text)

print("\n=====")
print("MULTIMODAL PIPELINE COMPLETED")
print("=====")

```

image.jpg file:



OUTPUT:

```
(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026> python visual_qa.py
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC
in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommen
d to use AFC in Chat.send_message_stream.
```

```
=====
VISUAL QA SYSTEM
=====
```

Question:

Describe what is visible in this image.
Identify the main objects and explain what is happening.

Answer:

Based on the image, here is a detailed description of what is visible and an explanation of the scene:

```
### **Visible Elements**
```

- The Motorcycle (Main Foreground Subject):**
 - A **Royal Enfield** motorcycle (specifically a 650 Twin model like the Continental GT or Interceptor 650) is parked at an angle facing toward the right.
 - Color & Design:** The fuel tank is gloss black with distinct yellow/gold and orange racing stripes and the "ROYAL ENFIELD" logo printed on the side.
 - Key Details:** It features shiny chrome engine casings, a large dual-exhaust system, silver spoked wheels, a round retro headlight, a single round chrome rearview mirror, and a black leather seat.
- The Background Wall:**
 - A textured, old stone brick wall covered in vibrant **graffiti artwork**.
 - Art Motifs:**
 - Multiple cartoonish, cloud-like shapes outlined in thick black with grey shading and orange highlights.
 - A colorful, detailed creature (resembling a dragon or serpent with scales) painted in shades of red, orange, yellow, and green stretching across the upper-left and center area.
 - A black floral outline on the far right and a signature at the bottom right that reads "Poornima".
- Ground and Lighting:**
 - The bike rests on a paved sidewalk/ground made of grey interlocking rectangular stones or brick tiles.
 - The lighting appears to be natural daylight, casting soft shadows beneath the motorcycle.
 - There is a watermark in the upper-left corner that reads "IAMABIKER.COM".

```
---
```

```
### **Main Objects & What is Happening**
```

Main Objects:

- The primary object is the **Royal Enfield motorcycle**.
- The secondary visual focus is the **graffiti wall** behind it.

What is Happening:

- This is a **static lifestyle or promotional motorcycle photograph**. The motorcycle is parked on its kickstand along an urban street, positioned deliberately in front of an artistic backdrop to highlight its retro-modern design and visual appeal. No dynamic movement or people are present in the shot.

```
=====
MULTIMODAL PIPELINE COMPLETED
=====
```

```
(.venv) PS C:\Users\Bhara\OneDrive\Desktop\Agentic-AI-lab internal-21-09-2026>
```

```
* History restored
```